CSC490 Assignment A4 - RL-ing Nanochat

Group 1 AGD-G
Avanti Tandon - 1010498695, Sark Asadourian - 1010726056,
Tianyu Luo - 1010130109, Tyson Caul - 1009951991

1 Part One: GRPO and RL Review

We start with a review of the reinforcement learning (RL) implementation for nanochat RL in nanochat is based on Group Relative Policy Optimization (GRPO) (DeepSeek-AI et al. , 2024). GRPO drops the value model to save memory and computation, which is highly efficient because a value model is typically the same size as the main policy model. To replace it, GRPO generates a group of multiple responses for each question, scores them using a reward model, and calculates an average baseline from these group scores. Nanochat's specific RL implementation is optimized for efficiency and for improving math performance on the GSM8K dataset. From this simplified use case, Karpathy introduces a few key modifications to standard GRPO that move the script closer to a basic REINFORCE algorithm.

First, the implementation removes the trust region and KL regularization penalty. Normally, this penalty prevents the model from generating answers that are too wildly different from the original base model. However, nanochat operates on objective math problems where answers are mechanistically checked for correctness. This makes "reward hacking" difficult, making the penalty unnecessary.

Second, the script relies strictly on on-policy training, meaning it only learns from the exact data it just generated rather than reusing old data. Because of this, there is no risk of the updated model drifting too far from the data-gathering model. This completely removes the need for PPO-style ratio and clipping functions, which are used to keep training stable.

Third, nanochat replaces sequence-level normalization with DAPO-style token-level normalization. Instead of calculating the learning updates for the entire answer as a whole, it averages the loss over the number of valid generated tokens. This ensures that updates remain mathematically stable and prevents unusually long answers from dominating the learning process.

Finally, the advantage calculation simplifies standard z-score normalization by only subtracting the mean reward $(r - \mu)$, rather than dividing by the standard deviation $(r - \mu)/\sigma$. Because GSM8K evaluation relies on binary rewards of exactly one or zero, the standard

deviation can be extremely small. Skipping this division prevents a tiny range of scores from triggering "gradient explosions", thereby improving overall training stability.

2 Part Two: SFT & Midtraining

Following an analysis of reinforcement learning implementations in the nanochat architecture, supervised fine-tuning (SFT) was conducted to enhance performance on ChatCORE, an evaluation suite for conversational tasks. SFT successfully increased the ChatCORE score from 0.0034 to 0.3678. From a machine learning perspective, this improvement occurs because SFT minimizes the cross-entropy loss between the model's outputs and high-quality human demonstrations, effectively aligning the model's response distribution with expected chatbot behavior.

To further target mathematical reasoning, additional datasets were introduced to improve performance on GSM8K. GSM8K serves as a reliable benchmark for basic arithmetic and is the primary dataset utilized for the subsequent reinforcement learning phase. Datasets with characteristics similar to GSM8K were selected, specifically those featuring grade-school mathematics paired with explicit chain-of-thought reasoning traces. It is worth noting that the data found lacked the use of python for arithmetic. While a limitation performance was still improved by adding these datasets and would likely further be improved with the use of python in traces as this is used in GSM8K

In the context of SFT data curation, there are two prevailing methodologies. The first paradigm argues that massive quantity overrides strict quality filtering, as models trained at scale are remarkably robust to noisy data. This approach is exemplified by the OpenMathInstruct-2 dataset (Toshniwal et al., 2024), which contains 14 million synthetically generated question-solution pairs. The second paradigm advocates for smaller, highly curated datasets enriched through diverse augmentation. For example, the MetaMathQA dataset (Yu et al., 2024) employs question bootstrapping, which rewrites prompts from multiple perspectives using forward and backward reasoning to maximize diversity gain and lower the perplexity of the target distribution.

Table 1: Evaluation Metrics Across Different SFT Configurations

Name	Runtime	ARC-C	ARC-E	GSM8K	HumanEval	MMLU	SpellingBee	ChatCORE
Base Line	1080	**0.4881**	0.6006	0.1250	0.1250	**0.3788**	1.0000	0.3678
MetaMathQA + OpenMathInstruct-2	1438	0.4821	0.6162	**0.1667**	0.0833	0.3723	1.0000	0.3685
MetaMathQA	1164	0.4761	0.5951	0.1250	**0.1667**	0.3759	1.0000	**0.3702**
OpenMathInstruct-2	1383	0.4753	**0.6275**	0.0417	0.1250	0.3747	1.0000	0.3561

These two paradigms were empirically compared by fine-tuning the model on 750,000 examples from the synthetic GSM8K OpenMathInstruct-2 dataset and 79,000 adversarial examples from MetaMathQA. The results, detailed in Table 1, indicate that MetaMathQA improved the ChatCORE score to 0.3702, compared to 0.3678 for the baseline SFT, while maintaining a GSM8K accuracy of 12.50%. Furthermore, MetaMathQA achieved the highest score on HumanEval at 16.67%. Conversely, OpenMathInstruct-2 degraded performance across the board, dropping ChatCORE to 0.3561 and GSM8K to 4.17%. From

an ML standpoint, the degradation caused by OpenMathInstruct-2 likely stems from an imbalance between the dataset's massive scale and the model's limited parameter capacity, where excessive verbosity in the chain-of-thought formatting can overwhelm smaller architectures.

When combining both MetaMathQA and OpenMathInstruct-2, the ChatCORE score reached 0.3684 and GSM8K accuracy peaked at 16.67%. Additionally, the ARC-Easy metric saw a slight improvement to 61.61%. This synergistic effect aligns with recent findings that mixing high-value examples with high-coverage data yields an optimal balance for easy-to-hard generalization, as noted in the FLAMES framework (Seegmiller et al., 2025). However, this combination also resulted in performance regressions on unrelated tasks, highlighting a classic alignment tax. Over-indexing on mathematical reasoning shifts the model's internal representations, causing forgetting that impairs general conversational capabilities and causes overfitting.

3 Part Three: Replicating our RL run with additional analysis

Model Justification and Baseline Comparison

Our Affine-Nanochat model with d24 gave us the following reward curves. We specifically utilized the d24 model because it had been previously trained by our team, providing a robust foundation for this phase. For the baseline reinforcement learning comparison, data was only available for Karpathy's d20 model. While the difference in layer count accounts for some performance variation, comparing these two runs yields valuable insights into the training dynamics.

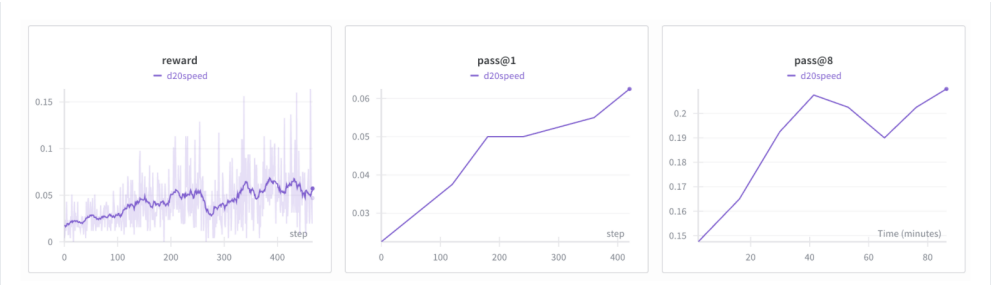

Figure 1: reward and eval curve for Karpathy's run

Another critical factor contributing to our model's performance is the expanded dataset used during our Supervised Fine-Tuning (SFT) phase. Unlike the baseline which utilized a smaller standard dataset, our d24 model benefited from a broader and more diverse set of training examples during SFT. This richer initialization directly explains the visual differences in the reward curves. Our d24 model exhibits a sharp initial spike in reward, indicating that the model rapidly aligned its pre-existing knowledge to the reward signal. The RL process did not have to teach the model mathematical reasoning from scratch.

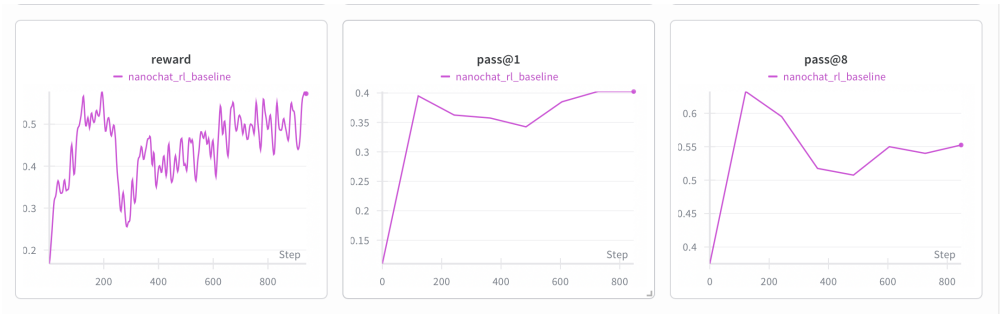

Figure 2: reward and eval curve with our sft

Instead, it was refining behaviors already present in the policy. As training continued, the curve became noisier, reflecting the increasing instability that often appears in RL once the easiest gains have been captured. In contrast, a model trained from scratch typically displays a smoother and more gradual climb.

Compared to Karpathy's original d20 run, our d24 run shows substantially higher task performance. Table 2 outlines this difference on the evaluation dataset.

Metric	Affine-Nanochat (d24)	Karpathy Original (d20)
Total examples	1319	1319
Correct answers	44.50%	7.58%

Table 2: Baseline RL performance comparison on GSM8K evaluation set

Exploratory Data Analysis

To better understand the mechanisms driving our model's performance, we conducted a targeted Exploratory Data Analysis focusing on four specific features: question length, unit tracking, code usage, and lexical diversity. We chose to analyze question and response length to measure how the model handles increased context size. Foundational research, such as by Wei et al. (2022), demonstrates that step-by-step reasoning significantly impacts performance on complex tasks.

We investigated traces use of consistent units, such as if the problem uses meters the trace also only uses meters, to evaluate the model's ability to maintain semantic consistency and logical anchoring throughout multi-step problems. Furthermore, we analyzed Python code usage to understand the model's reliance on external computation. Purely text-based arithmetic is a known weakness of language models. Research by Gao et al. (2023) shows that offloading math to a structured Python environment directly correlates with higher success rates. Finally, we measured lexical diversity to rule out reward hacking, ensuring the model was not simply being rewarded for repetitive stylistic phrasing.

Question and Response Length

We first examined whether the length of the question or the model response correlates with model performance. In the following tables, "Correct answers" refers to the percentage of queries within that specific word-count bin that resulted in a mathematically correct final answer.

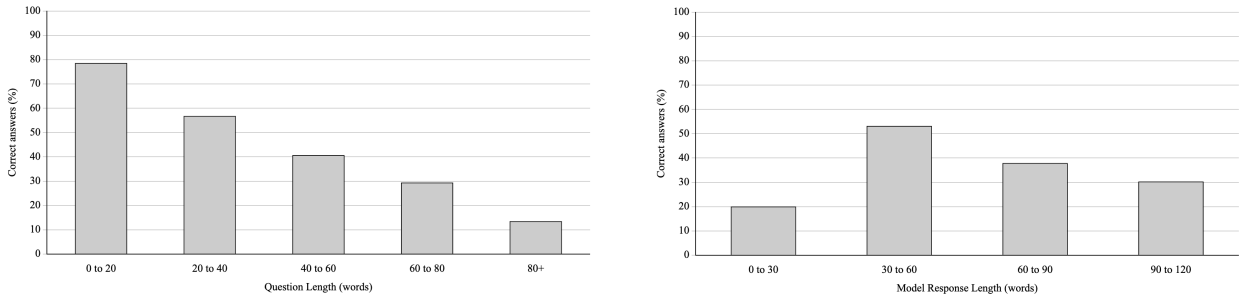

Figure 3: Plots of question length and response length with model accuracy.

Category	Avg Words	Avg Characters
Correct	40.7	211
Incorrect	50.7	263

Table 3: Average question length by prediction outcome

Figure 3 supports that the percentage of correct answers decreases significantly as question length increases. This implies that longer questions associate with higher difficulty, which the model struggles to process. Although Figure 3 suggests that model responses between 30 and 60 words are associated with a higher rate of correct answers, our analysis shows that questions answered correctly are shorter on average. Therefore, the apparent correlation between response length and correct answers may partially reflect differences in problem difficulty rather than the intrinsic benefit of shorter reasoning traces. The data does not support the conclusion that limiting sequence length would inherently improve performance.

Consistent Units

We analyzed whether the presence of units (such as "centimetres" or "days") in the model's reasoning is associated with numerical correctness. The intuition is that correctly tracking quantities and their associated units may help the model maintain semantic consistency during multi-step reasoning.

We define a response as *not needing units* if the reference (ground truth) response contains no units. We define a response as using *invalid units* if it mentions any unit which is not mentioned in the refence response. We define a response as *missing units* if it does not use all the units that the reference response does. Any response which uses exactly the same units as the reference is considered to use *valid units*. If a response is in multiple categories, it falls into whichever comes first in the following list:

1. No units needed
2. Invalid units
3. Missing units
4. Valid units

Category	Correct	Incorrect	Total	Accuracy
Valid units	293	367	660	44.4%
Missing units	57	103	160	11.8%
Invalid units	49	79	128	35.6%
No units needed	188	183	371	45.2%

Table 4: Distribution of unit usage among correct and incorrect predictions.

From the analysis in Table 4, several observations emerge:

- Questions that do not require units (because their ground truth answers do not include units) have the highest accuracy, possibly because they lack complexity.
- The model's accuracy is degraded when it introduces incorrect units.
- The model's accuracy is *significantly* degraded when it misses even a single unit from its answer.

This suggests that responses containing explicit unit information are more likely to produce the correct numerical answer. Although the relationship is not perfectly deterministic, the presence of units appears to provide a meaningful signal about the quality of the reasoning process.

Arithmetic Reasoning

The framework allows the model to generate Python code blocks to perform calculations. We analyzed how frequently these outputs appear and how they affect the rate of correct answers.

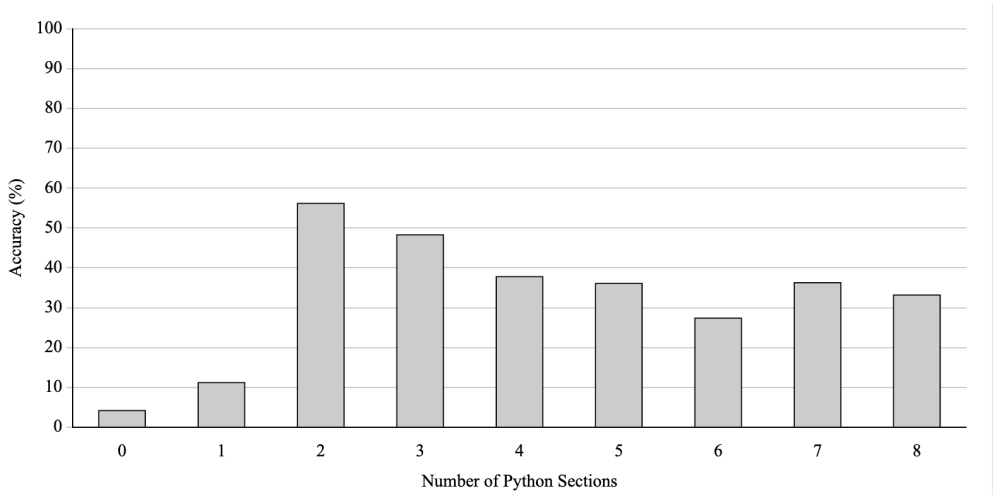

Figure 4: Number of python outputs in model response and the accuracy

The highest rate of correct answers occurs when the model produces two Python outputs. Conversely, problems with no Python outputs almost always result in incorrect answers. A key signal from this investigation is the consistency between intermediate Python outputs and the final predicted answer. We define consistency as occurring when the final numerical string generated by the model perfectly matches the exact output printed by the executed Python code block.

	Consistent	Inconsistent
Correct	580	7
Incorrect	570	162

Table 5: Consistency confusion matrix

The conditional probabilities confirm this trend, with $P(\text{correct} \mid \text{consistent}) = 0.504$ and $P(\text{correct} \mid \text{inconsistent}) = 0.041$. This result shows that reasoning inconsistency between the tool output and the model's text is a very strong indicator of failure.

Word Diversity

We analyzed lexical diversity using the type-token ratio (TTR).

Category	Average TTR
Correct responses	0.5205
Incorrect responses	0.5270

Table 6: Average TTR for correct and incorrect responses

Across the main mass of the dataset, the rate of correct answers remains largely flat. Lexical diversity appears to be a weak signal and a poor reward target because it is easy for the model to manipulate stylistically without improving the underlying mathematical reasoning.

Conclusion

Our analysis reveals that shorter questions, explicit unit tracking, and consistent intermediate Python outputs are the strongest indicators of successful reasoning in our model. Furthermore, the expanded SFT dataset provided a strong initialization that led to rapid initial reward gains during the RL phase. We also determined that lexical diversity is not a robust metric for correctness. These insights indicate that future reinforcement learning strategies for this architecture should prioritize reward signals that strictly enforce intermediate code execution and logical consistency over stylistic or length-based constraints.

4 Part Four: Introducing a more complex rewards system

To provide the model with more advanced reinforcement signals, we introduced two additional rewards:

- a penalty for failing to specify units where relevant, or using incorrect units
- a reward for using Python calls to compute the answer

Unit Consistency Reward

Rationale

We developed the unit reward on the premise that the reasoning and answer for a mathematical word question about, for example, days, hours, and dollars should include the words "day," "hour," and "dollar," and should not mention "pounds," "millimetres," or any unrelated units. A violation of this policy indicates that the assistant's response contains flawed reasoning or does not describe the solving process in sufficient detail. Per our data analysis above (Table 4), such a violation is associated with a severely decreased rate of correct answers. While we cannot infer causation from the correlation in the data analysis, we expected that enforcing correct units might improve numerical correctness or, at the very least, make answers more interpretable and semantically correct.

Method

We implemented the policy with a penalty proportional to the fraction of unit words from the question that were omitted from the response. On its own, this system would encourage the assistant to include arbitrary unit words in its response. Accordingly, we also assign the maximum penalty for including any irrelevant unit word, so guessing words is just as "bad" as including none. We define a fixed set of unit words that would be likely to appear in grade-school math, and use a normalization function that maps, for example, "metre," "meter," "metres," and "meters" to "metre" for comparison.

Formally, we define the reward as follows.

Define U as our set of unit words, U' as the set of their normal forms, and $norm : U \to U'$ as the normalization function (where $\text{norm}(S) = \{\text{norm}(s) : s \in S\}$ for any $S \subseteq U$). Define Q as the set of words in the question, R as the set of words in the assistant's response. Let $U_Q = \text{norm}(Q \cap U)$ and $U_R = \text{norm}(R \cap U)$. Let $\alpha_u \in [0, 1)$ be a reward multiplier. The reward is then

$$r_u(Q, R) = \begin{cases} -\alpha_u \max\left\{\frac{|U_Q \setminus U_R|}{|U_Q|}, \mathbb{I}\left[U_R \setminus U_Q \neq \emptyset\right]\right\} & Q \cap U \neq \emptyset \\ 0 & Q \cap U = \emptyset \end{cases}$$

The range of the reward is $[-\alpha_u, 0]$. If $\alpha_u = 0$, the unit reward system is disabled; in our experiments, we set this hyperparameter to $\alpha_u = 0.1$ to ensure the reward was meaningful but not dominating. Note that if there are no unit words in the question but there *are*

unit words in the response, the penalty is still zero; this allows the assistant to infer units without penalty in the case of an underspecified or strangely-worded question.

Results

When training with this adjusted reward (with $\alpha_u = 0.1$) added to the baseline reward, the accuracy on the GSM8k evaluation was 45.8%: 1.3 percentage points better than the baseline of 44.5%. See Table 7 for the full comparison.

Arithmetic Consistency Reward

Rationale

While the unit reward encourages correct quantity definitions and Karpathy's reward system encourages correct final output, it does not directly assess the model's reasoning output. To address this limitation, we introduce a arithemetic consistency reward that evaluates whether the final answer is supported by the reasoning.

As shown in the exploratory data analysis above, responses that do not make any calls to Python are very likely to fail, as are responses that use Python but no not use its output as the final answer. While we can't infer causation from this correlation, we expected that enforcing the use of Python output might improve correctness.

Background

The model offloads each arithmetic step to a Python tool call, in the form
`<|python_start|>9+10<|python_end|>`.
The output from the Python interpreter is then appended to the output, in the form
`<|output_start>19<|output_end|>`.
The model reads this Python output and outputs it again to free it from the Python tool call, which would ideally be removed if the output was sent to a user. This results in reasoning steps that look like the following:
`7+11 = <|python_start|>7+11<|python_end|><|output_start|>18<|output_end|>18`.

Method

This gives us an easy way to check whether the final answer is supported by the reasoning arithmetic: we just check whether the final answer is equal to the output of any Python call. Let P be the set of numbers output by Python calls (extracted by regular expression from `<|output_start|>...<|output_end|>` tags) and a be the final answer given by the model. The reward is then

$$r_c(P, a) = \begin{cases} 0 & a \in P \\ -\alpha_c & a \notin P \end{cases}$$

where $\alpha_c \in [0, 1)$ is a reward multiplier. We use a negative reward (penalty) so that rewards are comparable between the baseline and this model; in both, a reward of 1.0 indicates a

'perfect' answer, and because most responses already use Python (per data analysis), this penalty will be applied rarely.

This reward encourages responses that include explicit, tool-assisted arithmetic reasoning.

Results

When training with this adjusted reward (with $\alpha_c = 0.2$) added to the baseline reward, the accuracy on the GSM8k evaluation was 45.5%: one percentage point better than the baseline of 44.5%.

RL Ablations Results

Table 7: GSM8k Evaluation Metric Across Different Reward Configurations

Reward Configuration	α_u	α_c	GSM8k Accuracy
Baseline	0.0	0.0	44.5%
Units	0.1	0.0	45.8%
Consistency	0.0	0.2	45.5%
Both rewards	0.1	0.2	46.7%

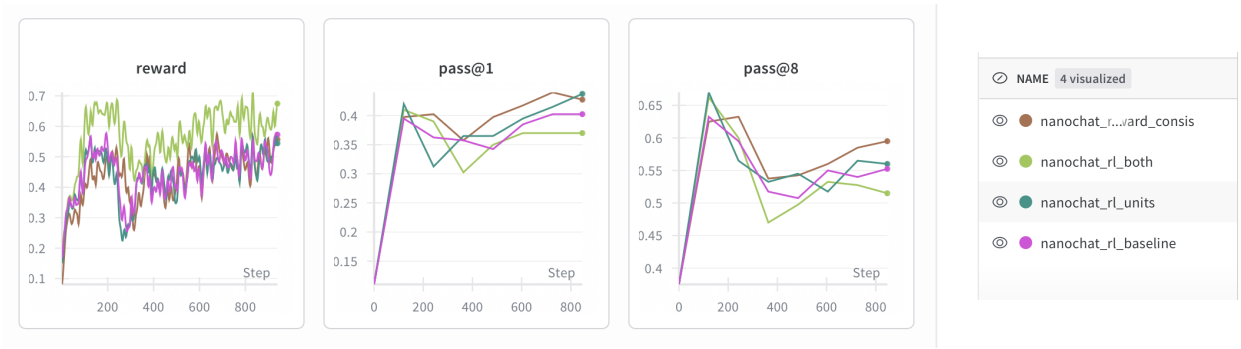

Figure 5: Comparison of the four reinforcement learning models used in our experiments.

The pass@8 metric is the percentage of questions for which at least one of the model's top 8 responses is correct. The pass@1 metric is the percentage of questions for the model's top (chosen) response is correct. Notice that the reward is higher for the *Both* model, despite both of our rewards being penalties; this indicates that the model is avoiding penalties and also answering correctly more often.

Comparison of Mistake Types

To understand how the additional rewards affect model behavior beyond overall accuracy, we compared the types of mistakes made by the baseline RL model and the three reward-augmented variants: RL with unit reward, RL with consistency reward, and RL with both rewards combined. We focus on two complementary views of error:

1. a coarse error taxonomy that separates arithmetic errors from missing final answers, and
2. a keyword-based categorization of the question types associated with wrong answers.

This analysis helps identify not only whether the rewards improve accuracy, but also how they change the model's failure modes.

Arithmetic errors vs. missing answers. We first simplify all incorrect predictions into two categories:

- Arithmetic error: the model produces a numeric final answer, but the number is incorrect.
- Missing answer: the model fails to produce a valid final numeric answer.

Figure 6 shows the distribution of these two mistake types across the four models. Several patterns are visible.

First, arithmetic errors remain the dominant failure mode for all runs. This suggests that most model failures are not due to formatting or termination issues, but rather due to incorrect numerical reasoning. In other words, even when the model produces a final answer, that answer is often mathematically wrong.

Second, the consistency reward model exhibits the largest proportion of missing-answer errors. This indicates that while the consistency reward can improve reasoning structure in some cases, it may also encourage longer or less decisive reasoning traces that fail to terminate with a final numeric answer. This is consistent with our earlier observation that the consistency reward increases the no-final-answer rate relative to the baseline.

Third, both the unit reward and the combined reward reduce the proportion of missing-answer errors compared to the consistency-only model. Among these, the combined reward model achieves the strongest overall balance: it improves accuracy while avoiding the sharp increase in missing-answer failures seen in the consistency-only model. This suggests that the unit reward partially stabilizes the output format and helps the model conclude with an explicit numeric answer.

Overall, this coarse mistake taxonomy shows that the additional rewards do not eliminate arithmetic mistakes, but they do change the balance between incomplete outputs and completed-but-wrong numerical reasoning. In particular, the consistency reward appears to shift some errors from wrong numbers toward incomplete outputs, while the combined reward recovers part of this loss.

Question-type distribution of wrong answers. To analyze error patterns at a finer level, we also classified questions by keyword into the following overlapping categories:

- percent/rate
- ratio/fraction
- counting/combinatorics
- difference/comparison

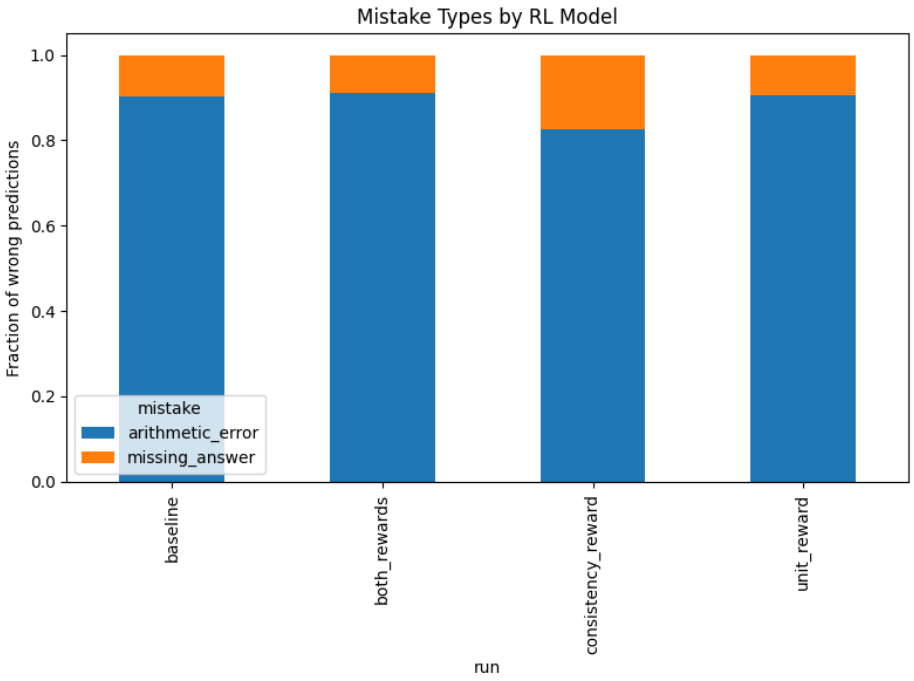

Figure 6: Stacked bar chart comparing arithmetic errors and missing-answer errors across the four RL models.

- time/work/motion
- money
- other

We then measured, for each model, the fraction of wrong answers whose questions belong to each category. Figure 7 visualizes these results as a heatmap.

Several trends emerge from the heatmap. Across all four runs, counting/combinatorics questions make up the largest share of wrong answers. This suggests that problems involving multiple quantities, aggregation, or combinational reasoning remain a persistent challenge regardless of reward design. Similarly, time/work/motion and money problems also occupy a relatively large share of the error distribution, indicating that multi-step arithmetic grounded in everyday quantities remains difficult for the model.

At the same time, the heatmap suggests that the reward-augmented models produce subtle shifts rather than dramatic changes in the category-level distribution of mistakes. The overall structure remains similar across runs, which implies that the additional rewards do not fundamentally change which semantic problem families are hardest. Instead, they appear to improve performance more through changes in reasoning quality within categories than by eliminating specific categories of errors altogether.

This is an important observation. It suggests that the added rewards are helping the model reason somewhat better on the same broad set of difficult question types, rather than making one problem family suddenly easy. In particular:

- the unit reward appears most naturally aligned with categories such as money and time/work/motion, where correct unit tracking is important;
- the consistency reward appears to influence the structure of reasoning, but does not substantially change which problem categories dominate the error set;
- the combined reward achieves the strongest overall accuracy, suggesting that these two reward signals are complementary even when their effects are not visually dramatic in the category heatmap.

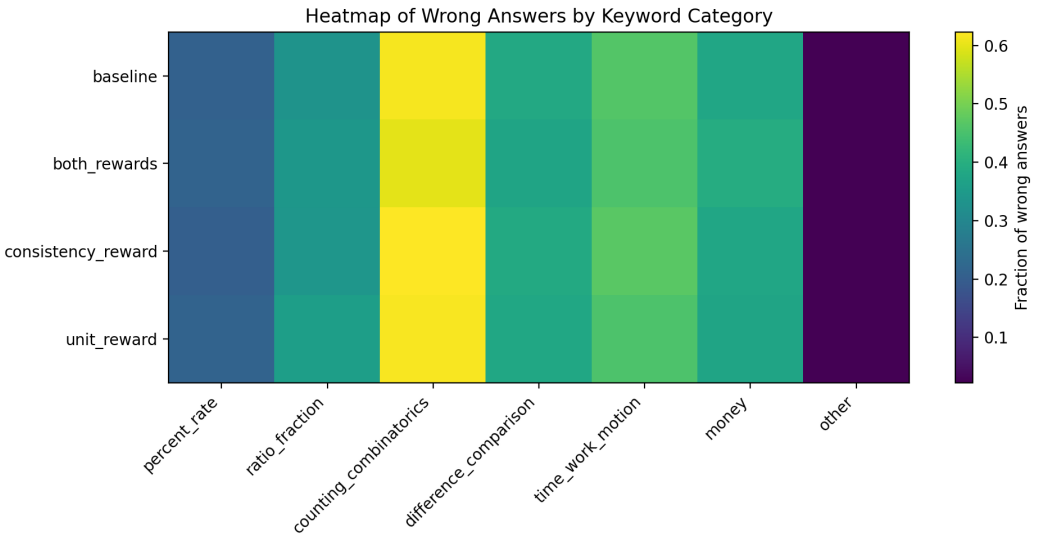

Figure 7: Heatmap showing the fraction of wrong answers associated with each keyword-based question category across the four RL models.

The main effect of the additional rewards reshaped the model's failure mode distribution rather than radically altered the set of problem categories it struggles with.

Summary

Model	Accuracy	Key Impact
Baseline RL	44.5%	Reference behavior
Unit Reward	45.8%	Improves arithmetic consistency
Consistency Reward	45.5%	More reasoning but more missing final answer
Both Rewards	46.7%	Best overall

Table 8: Summary of accuracy and mistake types across the four RL models.

Baseline RL achieved 44.5% accuracy. We believe the dominant failure mode was arithmetic error: in most incorrect cases, the model still produced a numeric answer, but the value was wrong. Missing final-answer predictions were much less common, indicating that baseline RL usually failed because of incorrect reasoning rather than formatting.

Adding the unit reward increased accuracy to 45.8% with little change in output formatting. This suggests that the reward slightly improved numerical stability, but did not substantially alter the overall error profile.

The consistency reward also improved accuracy, reaching 45.5%, but introduced more missing-answer failures, indicating a tradeoff between structured reasoning and stable output formatting.

The model trained with both rewards achieved the best overall result at 46.7%. Relative to the consistency-only model, it reduced missing-answer failures while also improving final accuracy, suggesting that the two reward signals were complementary. Across all runs, arithmetic mistakes remained the dominant failure mode, so the main limitation of the model is still numerical reasoning rather than answer formatting.

References

Gao, L., Maman, A., Banerjee, S., Goswami, V., Ku, A., Adams, S., . . . & Su, J. (2023). PAL: Program-aided Language Models. *arXiv preprint arXiv:2211.10435*. Link: `https://arxiv.org/abs/2211.10435`

DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., . . . Xie, Z. (2024). DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*. Link: `https://arxiv.org/abs/2405.04434`

Seegmiller, P., Mehta, K., Sahat, S., Tao, C., Oraby, S., Gupta, A., Chung, T., Bansal, M., & Peng, N. (2025). *FLAMES: Improving LLM Math Reasoning via a Fine-Grained Analysis of the Data Synthesis Pipeline*. Findings of the Association for Computational Linguistics: EMNLP 2025.

Toshniwal, S., Du, W., Moshkov, I., Kisacanin, B., Ayrapetyan, A., & Gitman, I. (2024). *OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data*. arXiv. `https://arxiv.org/abs/2410.01560`

Yu, J., Weller, A., Liu, Z., Liu, W., Yu, L., Jiang, W., Shi, H., Zhang, Y., Kwok, J. T., & Li, Z. (2024). *Metamath: Bootstrap your own Mathematical Questions for Large Language Models*. ICLR 2024. `https://arxiv.org/abs/2309.12284`

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*. Link: `https://arxiv.org/abs/2201.11903`